

Module Title

Cybersecurity and Networking

Internship Assignment Submission

OWASP Top 10:2025 Risk Identification

Student Name: Jiwosh Shrestha

Internship ID: JVL-CS-2026-8990

Role: Cybersecurity Intern

Duration: 3 months

Assignment Submission Date: Monday, 28th September 2026

Day:21 (Twenty-One)

Table of Contents

1.CROSS-SITE SCRIPTING (XSS) FUNDAMENTALS & SAFE LAB 3

1.1 PART A — FUNDAMENTALS 3

1.2 PART B — PORTSWIGGER LAB: REFLECTED XSS INTO HTML CONTEXT WITH NOTHING ENCODED 6

1.3 PART C — DEFENSIVE ANALYSIS 8

1.4 PART D — COMPARISON (SQL INJECTION VS CROSS-SITE SCRIPTING) 9

1.5 PART E — SHORT REFLECTION 10

1.CROSS-SITE SCRIPTING (XSS) FUNDAMENTALS & SAFE LAB

1.1 PART A — FUNDAMENTALS

1. What is XSS?

A client-side security vulnerability that allows an attacker to inject and execute arbitrary JavaScript in the context of a victim's web browser session.

2. Explain reflected XSS.

A non-persistent vulnerability where a malicious script payload sent in an HTTP request is immediately reflected back in the application's HTTP response without validation.

3. Explain stored XSS.

A persistent vulnerability where an injected script payload is permanently stored on the backend server (e.g., database) and executed whenever users view the stored data.

4. Explain DOM-based XSS.

A client-side XSS vulnerability where JavaScript code reads user input from an untrusted DOM source (like the URL) and writes it to an unsafe sink without encoding.

5. What is reflected input?

Data supplied by a client in an HTTP request that is directly copied or rendered back into the resulting HTTP response returned by the application server.

6. Why does input != XSS?

Reflection only proves data is echoed back; XSS requires an execution context where dangerous control characters (<, >) are rendered unencoded, allowing script parsing.

7. What is a canary value?

A unique, benign alphanumeric string used by security researchers to locate input reflection points within response bodies without triggering security alerts.

8. What is output encoding?

The defensive process of converting special characters into their safe, non-executable representation (e.g., converting '<' to '<') before rendering data in the browser.

9. Why match output context?

Different HTML contexts (HTML entity, attribute, JavaScript literal, CSS) use different parsers, requiring specific encoding schemes to safely isolate data from executable syntax.

10. HTML sanitization?

The process of parsing and inspecting incoming HTML markup to strip out executable code (scripts, event handlers) while allowing safe structural HTML tags like <b> or <i>.

11. Content Security Policy?

An HTTP response security header (CSP) that allows site administrators to restrict the sources from which scripts, styles, and dynamic resources can be loaded and executed.

12. CSP as defense-in-depth?

Because CSP misconfigurations or bypasses frequently occur; context-aware output encoding remains the primary defense required to prevent script execution at the source.

13. HttpOnly impact reduction?

The HttpOnly flag prevents client-side JavaScript from accessing sensitive session cookies via document.cookie, mitigating session hijacking even if XSS occurs.

14. HttpOnly remove XSS?

No. HttpOnly only protects session cookies from being read; the attacker can still perform unauthorized actions, manipulate the DOM, or launch phishing attacks within the user session.

1.2 PART B — PORTSWIGGER LAB: REFLECTED XSS INTO HTML CONTEXT WITH NOTHING ENCODED

1. Vulnerable feature?

The search functionality parameter on the product catalog.

2. Reflected or stored?

Reflected.

3. XSS type demonstrated?

Reflected Cross-Site Scripting (Reflected XSS).

4. Canary value used?

JVLXSS29

5. Where observed?

Inside the response body HTML heading tag: <h1>Search results for: JVLXSS29</h1>.

6. Burp Suite feature?

Burp Suite Repeater (used for manual request interception, parameter editing, and response comparison).

7. Training PoC payload?

<script>alert(1)</script>

8. Browser confirmation?

The native browser alert(1) popup dialog box.

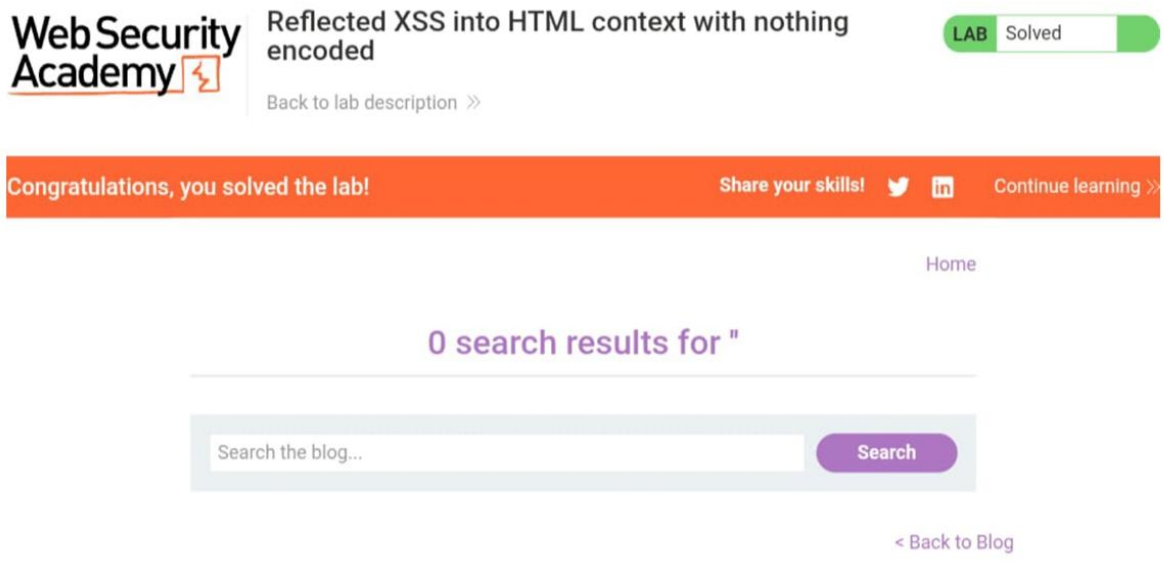
9. Why vulnerable?

User-supplied search input was directly reflected into the HTML output context without proper HTML entity output encoding.

10. Lab Solved Status?

Yes. PortSwigger displayed the confirmation banner: "Congratulations, you solved the lab!"

Training performed only in the provided PortSwigger lab.



1.3 PART C — DEFENSIVE ANALYSIS

1. Security risk?

Reflected Cross-Site Scripting (XSS), which allows attackers to execute arbitrary scripts in the victim's browser session.

2. Likely XSS category?

Reflected XSS.

3. Primary defensive control?

Context-aware HTML Entity Encoding on output (converting characters like '<' to '<', '>' to '>').

4. Sanitization required?

When an application legitimately needs to render rich HTML formatting (e.g., WYSIWYG editors) while neutralizing dangerous tags.

5. CSP additional protection?

By enforcing rules such as script-src 'self', blocking execution of inline scripts (<script>) and restricting untrusted script domains.

1.4 PART D — COMPARISON (SQL INJECTION VS CROSS-SITE SCRIPTING)

Parameter / Aspect: Interpreter

SQL Injection (SQLi): Database Engine Query Parser (MySQL, PostgreSQL)

Cross-Site Scripting (XSS): Client Web Browser Rendering Engine (HTML/JS)

Parameter / Aspect: Untrusted Input

SQL Injection (SQLi): Server-side database query logic and backend data access

Cross-Site Scripting (XSS): Client-side browser document object model (DOM) and sessions

Parameter / Aspect: Primary Defense

SQL Injection (SQLi): Parameterized Queries / Prepared Statements

Cross-Site Scripting (XSS): Context-Aware Output Encoding & CSP Headers

1.5 PART E — SHORT REFLECTION

This PortSwigger lab clearly highlighted the technical boundary between user DATA and executable BROWSER CONTENT. When an application accepts user input and reflects it directly into the HTML response without output encoding, the web browser cannot distinguish between passive text and active instructions. Injecting <script>alert(1)</script> caused the browser's HTML parser to transition into JavaScript execution mode, treating our data as executable code logic. Output encoding resolves this vulnerability by translating dangerous control characters into safe HTML entities, forcing the browser to treat input strictly as displayable data rather than dynamic commands. Ultimately, robust web application security depends on maintaining a strict structural separation between code compilation contexts and raw user input across all application layers.